

Troubleshooting Your Deploy

Sometimes, an app that runs fine locally might fail to deploy to Render at first. When this happens, it's almost always because of differences between your local development environment and the environment that Render uses to build and run your code.

These environmental differences might include:

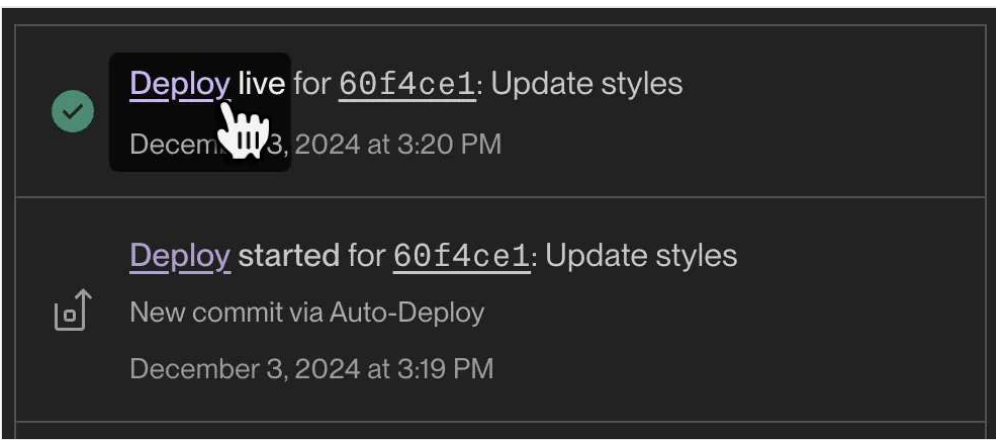
- The [version of your programming language](#)
- The values of important [environment variables](#)
- The availability of particular [tools and utilities](#)
- The versions of your project's dependencies

For recommended steps to resolve issues with your app's Render deploy, see below.

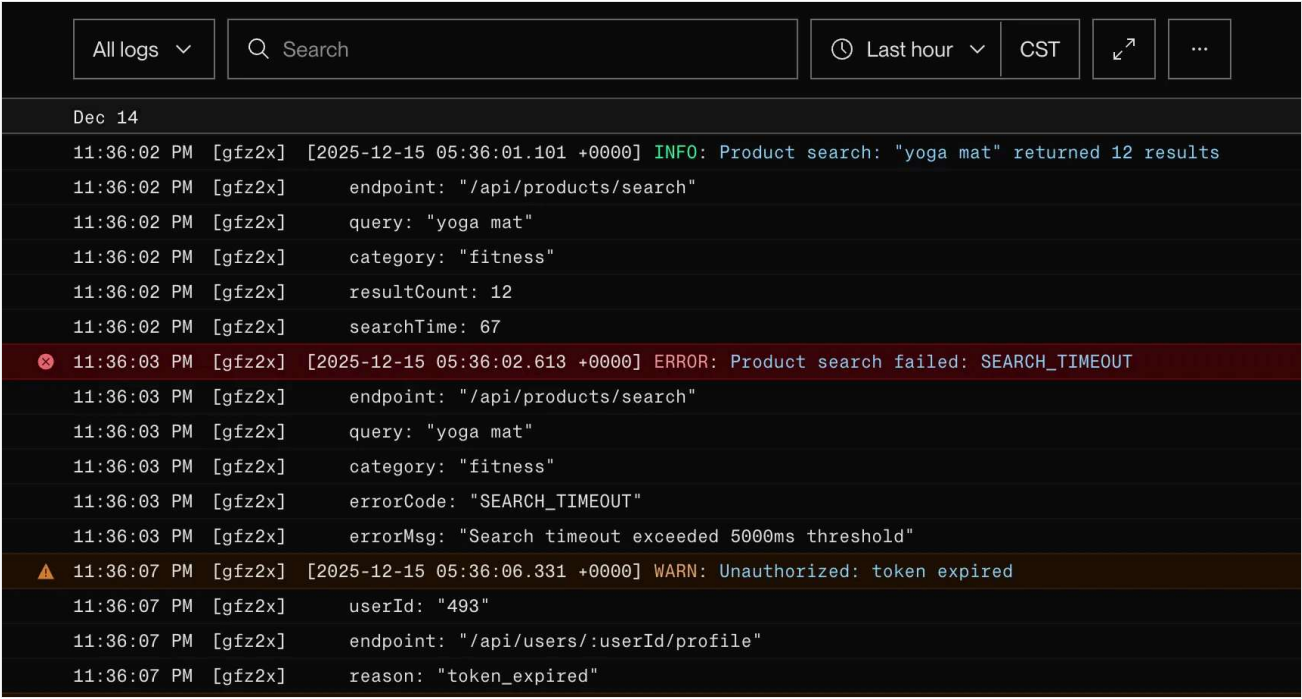
1. Check the logs

Whenever your app misbehaves in any way, **always check the logs first**. Logs are available in the [Render Dashboard](#):

- If a particular deploy fails, view its logs by clicking the word **Deploy** in your app's **Events** feed:



- If your running app encounters an error, open its **Logs** page to search and filter runtime logs with the explorer:



In either case, searching the log explorer for the word `error` can often direct you to a relevant log line. If the meaning of the full error message is unclear, try searching the web (or an individual site like Stack Overflow or [Discord](#) to help identify a root cause.

[Learn more about logging.](#)

2. Ensure matching versions and configuration

Render's runtime environment might use a different version of your app's programming language, or of an installed dependency. The values of certain [environment variables](#) might also differ from those on your local machine.

Check your app's configuration for the following:

Runtime mismatches

- While creating a service, you select the [runtime](#) that corresponds to your language (Node, Python, and so on). There's also a Docker runtime for projects that build from a Dockerfile or pull a prebuilt image.
- If you've selected an incorrect runtime for your app, the fastest fix is usually to create a *new* service with the correct runtime.
 - You can also change an existing service's runtime via Render Blueprints or the API. [See details.](#)

Version mismatches

- Each programming language has a *default* version on Render, which you can override to match your local machine's version. [See details.](#)
- Perform a fresh install of your project on your local machine to confirm that you're using exactly the dependency versions specified in your repository (such as in your `package-lock.json` file).

Configuration mismatches

- Your local machine might set [environment variables](#) as part of your app's start script, or via a file like `.env`. Make sure you're [setting necessary environment variables](#) on Render as well.
- When applicable, confirm that you've set necessary configuration to run your app in "production mode" (e.g., by setting `NODE_ENV` to `production`).
- To use any tools or utilities besides those [included by default](#) in Render's native runtimes, make that sure you install them as part of your app's [build command](#).
- Confirm that all of your app's dependencies are compatible with a Linux runtime and file system.
- [Check your logs](#) to confirm the following:
 - Your app's dependencies are all installed as expected.
 - Your service's start command runs and completes successfully.

Common errors

Build & deploy errors

Many first-time build and deploy errors are caused by one of the following issues:

Missing or incorrectly referenced resources

- **Module Not Found / `ModuleNotFoundError`**: Usually indicates one of the following:
 - A referenced dependency was not found (e.g., in your `package.json` or `requirements.txt` file).
 - A referenced file (such as `app.js` or `app.py`) was not found at a specified location.
 - If you're developing on Windows or another platform with a case-insensitive filesystem, make sure that all file paths, names, and extensions are cased correctly. You might need to check the contents of

your Git repo directly.

Language / dependency version conflicts

- **SyntaxError: Unexpected token '??='**: The app's Node.js version doesn't support the indicated operator or method.
- **The engine "node" is incompatible with this module. Expected version...**: The app's Node.js version doesn't work with the specified module.
- **requires Python >= 3.8**: A dependency is not compatible with the app's Python version.

Invalid configuration

- **Invalid build command**: The **command** that Render runs to install your project's dependencies and/or perform a build is missing or invalid.
 - This usually should match the command you run to build your app locally.
 - Common build commands include `npm install` (Node.js) and `pip install -r requirements.txt` (Python).
- **Invalid start command**: The **command** that Render runs to start your app is missing or invalid.
 - This usually should match the command you run to start your app locally.
 - Common start command formats include `npm start` (Node.js) and `gunicorn myapp:app` (Python).
- **Missing environment variables**: Some apps require certain environment variables to be set for them to build and start successfully.
 - Add environment variables to your app in the [Render Dashboard](#), or via a [render.yaml Blueprint file](#).
- **Missing Dockerfile CMD or ENTRYPOINT** : If you build and run your app from a [Dockerfile](#) , that file must include a `CMD` or `ENTRYPOINT` directive.
 - Render uses one of these directives to run your app after the build completes.
 - If you omit *both* of these directives, your deploy might appear to hang indefinitely in the [Render Dashboard](#).
- **Misconfigured health checks**: If you've added a health check endpoint to your app, Render uses it to verify that your app is responsive before marking it as live.
 - If the health check endpoint responds with an unexpected value (or doesn't respond at all), Render cancels your deploy.

Runtime errors

Many common runtime errors surface as **HTTP** error codes returned to your browser or other client. For errors returned to your browser, the Network panel of your browser's developer tools helps provide more details about the error.

Listed below are the most common error codes and some of their most common causes:

400 Bad Request

- A Django app doesn't include its associated [custom domain](#) in its [ALLOWED_HOSTS setting](#).

404 Not Found

- A static site has misconfigured [redirects and/or rewrites](#).
- A web service or static site has misconfigured its routing.
- A service is attempting to access a nonexistent file on disk. This might be because:
 - The file is no longer available because the service doesn't have a [persistent disk](#).
 - The service has provided the wrong path (such as by misspelling or incorrectly capitalizing a path component).
- A Django app is not correctly serving its [static files](#).

500 Internal Server Error

- A service has thrown an uncaught exception while responding to a request, possibly causing the service to crash or restart.
- A service is experiencing database connection issues, such as **SSL connection has been closed unexpectedly** .
 - In this case, setting `sslmode=require` and/or a setting up a [connection pool](#) can help.
- A service or database is overwhelmed, often by too many concurrent connections or constrained resources (such as CPU or RAM).
 - In this case, warnings about resource constraints usually appear in the service's [logs](#) and on the service's Events page in the [Render Dashboard](#).
 - To resolve, consider [scaling](#) your service to help alleviate load.

502 Bad Gateway

- A web service has misconfigured its [host and port](#).
 - Bind your host to `0.0.0.0` and optionally set the `PORT` environment variable to use a custom port (the default port is `10000`).
- A newly added [custom domain](#) is not yet redirecting to its web service.
 - In most cases this resolves within a few minutes, but it might take up to an hour.
- A Node.js web service is experiencing intermittent timeouts or **Connection reset by peer** errors. Try increasing the values for `server.keepAliveTimeout` and `server.headersTimeout` (such as to `120000` for 120 seconds).
- A service is experiencing **WORKER** , **SIGKILL** , or **SIGTERM** warnings (e.g., **[CRITICAL] WORKER TIMEOUT**).
 - Consider increasing your timeout values and worker limits (e.g., via the [gunicorn timeout parameter](#)).

Copy page

Troubleshooting Your Deploy

- 1. Check the logs
- 2. Ensure matching versions and configuration

Common errors

Build & deploy errors

Runtime errors

When to contact support

Did this page help?

When to contact support

Render's support team is available and happy to assist with issues that are specific to the capabilities, conventions, and underlying infrastructure of our platform.

Our support team cannot assist with more general development issues like the following:

- Debugging of application code
- Software design and architecture
- Performance optimization
- Programming nuances specific to a particular library or framework

For help with issues like these, please consult sites and services that specialize in these forms of assistance.

Compute

Static sites

Web services

Private services

Background workers

Cron jobs

Workflows

Deploys

How deploys work

Supported languages

Build pipeline

Deploy hooks

Monorepo support

Troubleshooting deploys

Previewing changes

Single-service previews

Preview environments (multi-service)

Docker deploys

Overview

Deploy a registered image

Using secrets

Runtime

Native runtimes

Environment variables & secrets

Default environment variables

Git providers

GitHub

GitLab

Bitbucket

Workflows

Intro to workflows

Tutorial